

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



COMPUTER NETWORK (CO3093)

Assignment 1

“Implement HTTP server and chat application”

Instructor(s): ThS. Bùi Xuân Giang

Students: Trần Vũ Đình Huy - 2352415
Nguyễn Tấn Đạt - 2352234
Vũ Đào Thanh Tùng - 2353298
Võ Quốc Phong - 2352913

HO CHI MINH CITY, NOVEMBER 2025



Contents

Member list & Workload	2
1 Giới thiệu	3
2 Phân tích yêu cầu bài toán	3
2.1 Task 1A – Authentication qua HTTP POST	3
2.2 Task 1B – Cookie-based Access Control	3
2.3 Task 2.2 – Peer-to-Peer Communication	4
3 Kiến trúc hệ thống	4
3.1 Sơ đồ tổng quan	4
4 Phân tích mã nguồn hệ thống	4
4.1 Tổng quan kiến trúc	4
4.2 Tập tin <code>start_app.py</code>	5
4.3 Lớp <code>HttpAdapter</code> – web server dựa trên socket	5
4.4 Lớp <code>Request</code> – phân tích gói tin HTTP	6
4.5 Lớp <code>Response</code> – xây dựng gói tin HTTP	6
4.5.1 Hàm <code>build_response</code>	6
4.5.2 Hàm <code>build_json_response</code>	7
4.5.3 Hàm <code>build_file_response</code>	7
4.6 Lớp <code>WeApRous</code> – framework định tuyến mini	8
4.6.1 Đăng ký route bằng decorator	8
4.6.2 Phương thức <code>handle_request</code>	8
4.7 File <code>apps/app.py</code> – logic nghiệp vụ	8
4.7.1 API <code>/login</code> – Task 1A: Xử lý đăng nhập	9
4.7.2 Kiểm soát truy cập bằng cookie – Task 1B	10
4.7.3 Các API Task 2.2 – quản lý peer và gửi tin nhắn	10
4.8 Giao diện web: <code>login.html</code> , <code>index.html</code> , <code>chat.html</code>	10
4.8.1 Trang <code>login.html</code>	10
4.8.2 Trang <code>index.html</code> – màn hình welcome	11
4.8.3 Trang <code>chat.html</code> – giao diện chat Task 2.2	11
5 Kiểm thử và kết quả	11
5.1 Test Case 1 – Login đúng	11
5.2 Test Case 2 – Login sai	12
5.3 Test Case 3 – Load ảnh <code>/static/images/welcome.jpg</code>	12
5.4 Test Case 4 – 2 peer chat trực tiếp	13
5.5 Test Case 5 – Broadcast	13
6 Kết luận	14



Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Nguyễn Tấn Đạt	2352234	2.1	100%
2	Trần Vũ Đình Huy	2352415	2.2	100%

Table 1: Member list & workload

1 Giới thiệu

Bài tập lớn môn **Computer Networks** yêu cầu sinh viên xây dựng một ứng dụng Web lai (Hybrid Web Application) với backend hoạt động trực tiếp trên **TCP**, xử lý **HTTP thủ công**, kết hợp với UI sử dụng HTML/CSS/JS. Sinh viên không được phép sử dụng các thư viện Web framework như **Flask**, **Django**, **FastAPI**, mà thay vào đó phải tự dựng:

- Bộ phân tích HTTP request ở mức byte
- Bộ tạo HTTP response (header + body)
- Cơ chế định tuyến (Route Handler)
- Static File Server
- Authentication sử dụng Cookie
- Giao thức Chat Peer-to-Peer trên TCP
- Web UI cho login, hiển thị ảnh, chat realtime

Mục tiêu của bài tập lớn không chỉ là hoàn thiện chức năng, mà còn giúp sinh viên hiểu cách hoạt động thực sự của HTTP/TCP, giúp hình dung rõ ràng toàn bộ chu trình Request-Response theo mô hình OSI.

2 Phân tích yêu cầu bài toán

2.1 Task 1A – Authentication qua HTTP POST

Yêu cầu:

- Nhận yêu cầu POST /login
- Parse toàn bộ request gồm:
 - Request-Line
 - Headers (text-based)
 - Body (JSON)
- Kiểm tra username/password
- Gửi về JSON + Set-Cookie

2.2 Task 1B – Cookie-based Access Control

Khi user truy cập /, backend phải:

- Đọc cookie từ header request
- Kiểm tra tính hợp lệ (auth=true, sessionid tồn tại)
- Load file `index.html` từ thư mục `www/`
- Hoặc trả về 401 Unauthorized

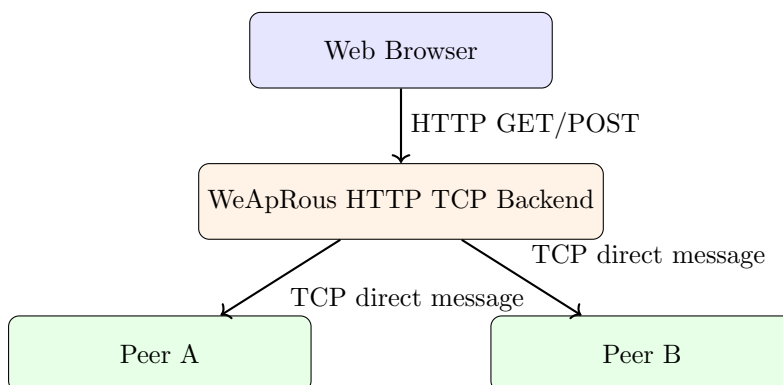
2.3 Task 2.2 – Peer-to-Peer Communication

Yêu cầu nâng cao:

- Người dùng tự khai báo username, địa chỉ IP, port
- Server lưu danh sách peer đang online
- Cho phép:
 - Nhắn tin trực tiếp đến 1 peer (TCP)
 - Gửi broadcast tới tất cả peers
- UI hiển thị danh sách online và message realtime

3 Kiến trúc hệ thống

3.1 Sơ đồ tổng quan



Kiến trúc chia làm 3 thành phần chính:

1. **HTTP Adapter** — nhận TCP, phân tích bytes thành HTTP request
2. **Application Layer (app.py)** — xử lý logic, route
3. **Response Engine** — xây dựng HTTP response, file server

4 Phân tích mã nguồn hệ thống

Trong mục này, nhóm trình bày chi tiết cấu trúc mã nguồn của dự án, vai trò của từng module chính và cách các thành phần tương tác với nhau để hiện thực yêu cầu của Assignment 1 (HTTP server, cơ chế đăng nhập, kiểm soát truy cập bằng cookie và ứng dụng chat kiểu peer-to-peer).

4.1 Tổng quan kiến trúc

Mã nguồn được tổ chức theo ba lớp chính:

- **Lớp giao thức thấp (daemon):** hiện thực web server dựa trên socket TCP, parse/generate gói tin HTTP. Các file chính: `httpadapter.py`, `request.py`, `response.py`, `backend.py`, ...

- **Lớp framework mini (WeApRous):** cung cấp cơ chế định tuyến (routing) giống Flask: ánh xạ cặp (method, path) sang hàm xử lý. File chính: `weaprous.py`.
- **Lớp nghiệp vụ (applications):** hiện thực các chức năng yêu cầu của đề bài: đăng nhập, quản lý session, API chat. File chính: `apps/app.py`. Phần giao diện người dùng được đặt trong thư mục `www/` (HTML) và `static/` (CSS, image).

Luồng xử lý tổng quát:

1. Người dùng chạy lệnh `python start_app.py` để khởi động server.
2. `HttpAdapter` lắng nghe kết nối TCP trên cổng được chỉ định, đọc request HTTP raw và xây dựng đối tượng `Request`.
3. Đối tượng `Request` được chuyển cho đối tượng `WeApRous` để tìm route tương ứng; sau đó `Response` sẽ xây dựng gói tin HTTP trả về.
4. Nếu đường dẫn trùng với một API (ví dụ `/login`, `/broadcast-peer`) thì `Response` trả về JSON. Nếu không, server cố gắng phục vụ file tĩnh từ thư mục `www/` hoặc `static/`.

4.2 Tập tin `start_app.py`

Tập tin `start_app.py` là điểm vào (entry point) của ứng dụng. Nhiệm vụ chính:

- Đọc tham số dòng lệnh `-server-ip` và `-server-port` bằng thư viện `argparse`.
- Khởi tạo đối tượng `WeApRous` (được định nghĩa trong `apps/app.py`) và gắn nó vào lớp `HttpAdapter`.
- In ra màn hình các thông tin hướng dẫn test:
 - URL backend: `http://0.0.0.0:8000`
 - Đường dẫn mở giao diện chat: `http://127.0.0.1:8000/chat.html`
 - Tài khoản mặc định: `admin / password`.
- Gọi hàm `serve_forever()` của `HttpAdapter` để bắt đầu vòng lặp chấp nhận kết nối.

4.3 Lớp `HttpAdapter` – web server dựa trên socket

File `daemon/httpadapter.py` hiện thực một web server đơn giản bằng Python socket, thay thế cho các framework quen thuộc như Flask hay Django.

Các bước xử lý chính trong `HttpAdapter`:

1. Tạo socket TCP, bind vào cặp (`server_ip`, `server_port`) và gọi `listen()`.
2. Trong vòng lặp, gọi `accept()` để nhận kết nối mới, sinh ra một socket con tương ứng với mỗi client.
3. Đọc dữ liệu từ socket con cho đến khi gặp chuỗi `\r\n\r\n` (kết thúc phần header). Nếu tồn tại header `Content-Length` thì tiếp tục đọc thêm phần thân (body) tương ứng.
4. Tạo đối tượng `Request` từ chuỗi HTTP raw vừa đọc, sau đó chuyển cho lớp `WeApRous` để xử lý:

```
1 request = Request(raw_http)
2 response_bytes = app.handle_request(request)
3 conn.sendall(response_bytes)
4 conn.close()
```

Như vậy, `HttpAdapter` chỉ chịu trách nhiệm ở lớp giao thức TCP/HTTP thô, không chứa logic nghiệp vụ.

4.4 Lớp Request – phân tích gói tin HTTP

File `daemon/request.py` chịu trách nhiệm phân tích request HTTP thành các trường có cấu trúc.

Các thông tin quan trọng được trích ra:

- `self.method`: phương thức HTTP ("GET", "POST", ...)
- `self.path`: đường dẫn, ví dụ `/login`, `/chat.html`, `/static/images/welcome.jpg`.
- `self.headers`: từ điển chứa các header (host, user-agent, content-type,...).
- `self.cookies`: từ điển cookie, được parse từ header `Cookie`.
- `self.body`: phần nội dung request (dạng chuỗi).

Nếu `Content-Type` là `application/json`, module này còn hỗ trợ parse body thành đối tượng JSON để việc xử lý ở lớp trên trở nên thuận tiện hơn.

Trong quá trình routing, lớp `WeApRous` sẽ gắn thêm thuộc tính `hook` vào `Request` để trở tới hàm xử lý tương ứng (nếu tồn tại).

4.5 Lớp Response – xây dựng gói tin HTTP

File `daemon/response.py` chịu trách nhiệm xây dựng gói tin HTTP trả về cho client dưới dạng bytes. Đây là một phần quan trọng vì toàn bộ header, body, trạng thái HTTP đều phải được tự tay định dạng.

4.5.1 Hàm `build_response`

Hàm `build_response(self, request)` là điểm vào chung:

```
1 def build_response(self, request):
2     if request is None:
3         return self.build_error_response(500, "Empty request object")
4
5     # Case 1: request c hook -> API JSON (route của WeApRous)
6     if getattr(request, "hook", None):
7         return self.build_json_response(request)
8
9     # Case 2: không c route -> phục vụ file tĩnh (HTML, CSS,
10    image, ...)
11    return self.build_file_response(request)
```

Như vậy, server phân biệt hai loại request:

- Request tới các API của ứng dụng (đã được đăng ký bằng decorator `@app.route`) sẽ được trả về dạng JSON.
- Các request khác (ví dụ truy cập `/login`, `/chat.html` hay hình ảnh) sẽ được coi là yêu cầu file tĩnh và được phục vụ từ thư mục `www/` hoặc `static/`.

4.5.2 Hàm `build_json_response`

Hàm này đảm nhiệm việc gọi handler tương ứng và đóng gói kết quả thành một response JSON:

- Gọi `request.hook(request=request)` để thực thi logic nghiệp vụ.
- Hỗ trợ nhiều kiểu giá trị trả về khác nhau:
 - `(status_code, data)`
 - `(status_code, data, cookies_dict)` – dùng cho trường hợp login, cần gửi Set-Cookie.
 - Chỉ `data` – mặc định `status_code = 200`.
- Chuyển `data` sang dạng JSON bằng `json.dumps` và tính toán `Content-Length`.
- Ánh xạ mã trạng thái (200, 401, ...) sang chuỗi tương ứng ("OK", "Unauthorized", ...) để tạo dòng trạng thái HTTP/1.1 200 OK.
- Nếu handler trả về từ điển `cookies`, hàm sẽ chèn thêm các dòng Set-Cookie vào phần header HTTP.

4.5.3 Hàm `build_file_response`

Hàm `build_file_response(self, request)` được dùng để phục vụ file tĩnh (HTML, CSS, JavaScript, hình ảnh...). Các bước:

1. Chuẩn hoá đường dẫn:
 - Nếu path trống hoặc `"/` thì ánh xạ về `/index.html`.
 - Nếu đoạn cuối không chứa dấu chấm (ví dụ `/login`) thì tự động thêm phần mở rộng `.html` $\Rightarrow$ `/login.html`.
2. Nếu đường dẫn bắt đầu bằng `/static/` thì file được tìm trong thư mục `static/`, ngược lại tìm trong `www/`.
3. Kiểm tra sự tồn tại của file; nếu không tìm thấy thì trả về lỗi 404 bằng hàm `build_error_response`.
4. Đọc nội dung file dạng nhị phân, xác định loại MIME bằng `mimetypes.guess_type`, sau đó xây dựng đầy đủ header:

```
1 status_line = "HTTP/1.1 200 OK\r\n"
2 headers = f"Content-Type: {mime_type}\r\n"
3 headers += f"Content-Length: {len(content)}\r\n\r\n"
```

Nhờ đó, cùng một server có thể vừa trả HTML (`index.html`, `login.html`, `chat.html`), vừa trả file ảnh (`welcome.jpg`) và stylesheet.

4.6 Lớp WeApRous – framework định tuyến mini

File `daemon/weaprou.py` định nghĩa lớp `WeApRous`, đóng vai trò là một mini-framework để định tuyến request tương tự Flask.

4.6.1 Đăng ký route bằng decorator

```
1 class WeApRous:
2     def __init__(self):
3         self.routes = {} # key: (method, path) -> handler
4
5     def route(self, path, methods=["GET"]):
6         def decorator(func):
7             for m in methods:
8                 self.routes[(m, path)] = func
9             return func
10        return decorator
```

Nhờ đó, trong file `apps/app.py` có thể viết:

```
1 @app.route("/login", methods=["POST"])
2 def login(request):
3     ...
```

4.6.2 Phương thức `handle_request`

Khi `HttpAdapter` nhận được một `Request`, nó sẽ gọi:

```
1 def handle_request(self, request: Request):
2     key = (request.method, request.path_without_query)
3     request.hook = self.routes.get(key)
4     resp = Response()
5     return resp.build_response(request)
```

Nếu tồn tại handler tương ứng trong `routes`, thuộc tính `hook` của `Request` sẽ được gán và `Response` sẽ xử lý theo nhánh JSON; nếu không có, request sẽ được coi là yêu cầu truy xuất file tĩnh.

4.7 File `apps/app.py` – logic nghiệp vụ

Toàn bộ logic theo yêu cầu của đề bài (Task 1A, 1B và Task 2.2) được hiện thực trong file `apps/app.py`. Một số cấu trúc dữ liệu toàn cục được sử dụng:

- **SESSIONS**: ánh xạ `sessionid` → thông tin người dùng (username, thời điểm tạo session).
- **PEERS**: danh sách các peer đã đăng ký trong hệ thống chat, mỗi phần tử lưu địa chỉ IP và cổng TCP.
- **MESSAGES**: lưu lịch sử tin nhắn theo từng kênh (general, direct chat).

4.7.1 API /login – Task 1A: Xử lý đăng nhập

API /login sử dụng phương thức POST với nội dung JSON:

```
1 @app.route("/login", methods=["POST"])
2 def login(request=None, body=""):
3     try:
4         data = json.loads(request.body)
5         username = data.get("username", "")
6         password = data.get("password", "")
7     except:
8         return (401, {
9             "status": "unauthorized",
10            "message": "Invalid JSON format in request body"
11        })
12
13    if username == "admin" and password == "password":
14        sessionid = generate_sessionid()
15        SESSIONS[sessionid] = {
16            "username": username,
17            "created_at": time.time(),
18        }
19
20        data = {
21            "status": "logged_in",
22            "message": "Authentication successful",
23            "username": username,
24            "auth": True,
25        }
26        cookies = {
27            "auth": "true",
28            "sessionid": sessionid,
29            "username": username,
30        }
31        return (200, data, cookies)
32    else:
33        return (401, {
34            "status": "unauthorized",
35            "message": "Invalid username or password",
36            "auth": False,
37        })
```

Cơ chế hoạt động:

- Server đọc JSON từ body, trích username và password.
- Nếu trùng với thông tin yêu cầu của đề (admin/password), server sinh một sessionid ngẫu nhiên, lưu vào SESSIONS.
- Response trả về gồm:
 - Thân JSON thông báo đăng nhập thành công.
 - Header Set-Cookie thiết lập các cookie auth, sessionid và username để trình duyệt tự động gửi trong các request tiếp theo.
- Nếu thông tin sai, server trả về mã lỗi 401 Unauthorized cùng thông điệp phù hợp.

4.7.2 Kiểm soát truy cập bằng cookie – Task 1B

Để minh họa cơ chế kiểm soát truy cập bằng cookie, hệ thống triển khai một route dùng để kiểm tra trạng thái đăng nhập. Ý tưởng chung:

1. Đọc cookie `auth`, `sessionid`, `username` từ đối tượng `Request`.
2. Nếu `auth=true` và `sessionid` tồn tại trong `SESSIONS`, coi như người dùng đã đăng nhập hợp lệ.
3. Nếu cookie sai hoặc session không tồn tại, server trả về mã lỗi 401 cùng thông điệp tương ứng.

Qua đó, Task 1B cho thấy cách server có thể sử dụng cookie để giới hạn quyền truy cập tài nguyên.

4.7.3 Các API Task 2.2 – quản lý peer và gửi tin nhắn

Tùy theo đề cụ thể, các API chính phục vụ Task 2.2 bao gồm:

- `/submit-info`: nhận thông tin peer (username, IP, port TCP) và lưu vào `PEERS`.
- `/add-list`: thêm peer vào danh sách để phục vụ mục đích demo.
- `/get-list`: trả về danh sách peer đang đăng ký.
- `/connect-peer`: tạo yêu cầu kết nối direct TCP từ peer A đến peer B.
- `/broadcast-peer`: gửi tin nhắn broadcast đến tất cả peer.
- `/send-peer`: gửi tin nhắn trực tiếp giữa hai peer.

Các API này thể hiện mô hình “hybrid”: HTTP dùng để điều phối phiên chat và quản lý peer, trong khi việc truyền tin nhắn thực tế giữa các peer được tiến hành qua kết nối TCP trực tiếp.

4.8 Giao diện web: `login.html`, `index.html`, `chat.html`

4.8.1 Trang `login.html`

Trang `login.html` hiện thực giao diện đăng nhập cho Task 1A. Giao diện được thiết kế với:

- Card đăng nhập nằm giữa màn hình, nền gradient tối, input cho `username` và `password`.
- Sử dụng API `fetch` để gửi request `POST /login` với nội dung JSON:

```
1 fetch("/login", {
2   method: "POST",
3   headers: {"Content-Type": "application/json"},
4   body: JSON.stringify({ username, password }),
5   credentials: "include" // de trình duyệt gửi/nhận cookie
6 })
7   .then(res => res.json())
8   .then(data => {
9     if (data.auth) {
10       // hiển thông báo success, sau đó redirect về "/"
11       window.location.href = "/";
```

```
12         } else {  
13             // hien loi ra giao dien  
14         }  
15     });
```

Việc sử dụng `credentials: "include"` đảm bảo browser chấp nhận cookie `Set-Cookie` do server gửi về.

4.8.2 Trang `index.html` – màn hình welcome

Sau khi đăng nhập thành công, trên trang login sẽ chuyển hướng người dùng đến đường dẫn gốc `/`. Lúc này:

- Server phục vụ file `www/index.html` dưới dạng file tĩnh.
- Nội dung trang đơn giản: hiển thị hình ảnh `welcome.jpg` với dòng chữ `bksysnet@hcmut` ở giữa màn hình, đóng vai trò màn hình welcome của hệ thống.

Trang này giúp minh họa việc kết hợp giữa kiểm soát truy cập bằng cookie và phục vụ nội dung HTML sau khi đăng nhập.

4.8.3 Trang `chat.html` – giao diện chat Task 2.2

Trang `chat.html` cung cấp giao diện chat :

- Khu vực bên trái: cho phép nhập username, IP, cổng TCP và nút *Login / Register Peer*. Khi nhấn, gọi các API như `/submit-info`, `/add-list` để đăng ký peer với server.
- Khu vực chính: hiển thị danh sách tin nhắn; người dùng có thể gửi tin nhắn broadcast hoặc tin nhắn trực tiếp tới một peer cụ thể thông qua các API `/broadcast-peer` và `/send-peer`.
- Tầng dưới: JavaScript định kỳ hoặc sau mỗi lần gửi tin sẽ gọi `/channel/messages` để cập nhật lịch sử chat và render lại giao diện.

Giao diện này minh họa cách kết hợp HTTP API với kết nối TCP direct để xây dựng một ứng dụng chat đơn giản nhưng có đầy đủ chức năng: đăng ký peer, gửi broadcast, gửi direct message.

Tóm lại, mã nguồn của hệ thống đã hiện thực một mini web server đầy đủ chức năng trên nền socket TCP, hỗ trợ định tuyến, xử lý JSON, cookie và file tĩnh mà không phụ thuộc vào framework web có sẵn. Trên nền đó, các yêu cầu của bài tập lớn (đăng nhập, kiểm soát truy cập bằng cookie, và ứng dụng chat peer-to-peer) được tổ chức thành các module rõ ràng, dễ mở rộng và thuận lợi cho việc kiểm thử.

5 Kiểm thử và kết quả

5.1 Test Case 1 – Login đúng

- Input: admin/password
- Expect: 200 OK + cookie + redirect /

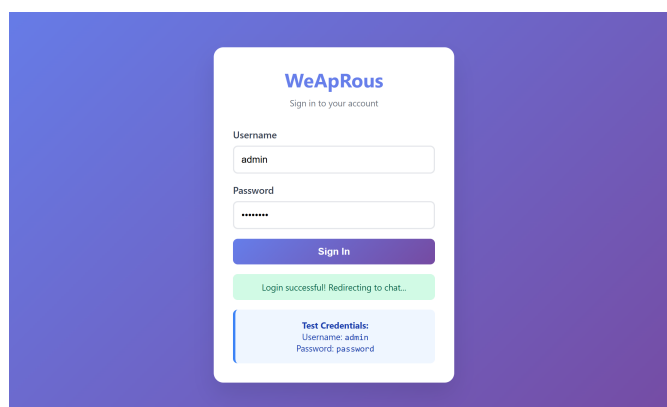


Figure 1: Giao diện đăng nhập của hệ thống sau khi đăng nhập thành công

5.2 Test Case 2 – Login sai

- Expect: 401 Unauthorized

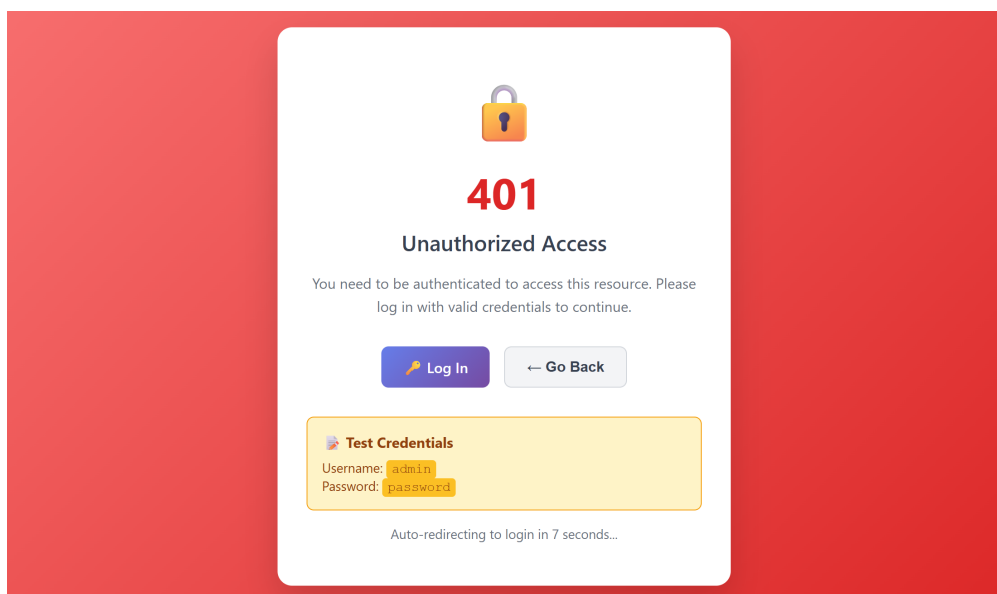


Figure 2: Giao diện hiển thị ảnh sau khi đăng nhập fail

5.3 Test Case 3 – Load ảnh /static/images/welcome.jpg

- Expect: Hiển thị bình thường (đã sửa lỗi path)



Figure 3: Giao diện hiển thị ảnh sau khi đăng nhập thành công

5.4 Test Case 4 – 2 peer chat trực tiếp

User A gửi tin → User B nhận được ngay.

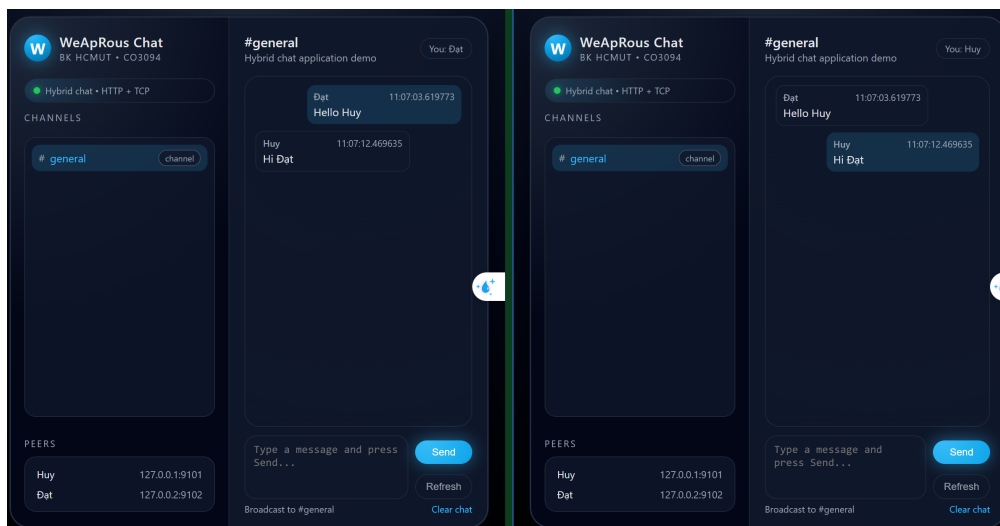


Figure 4: A gửi tin nhắn cho B

5.5 Test Case 5 – Broadcast

Tin nhắn gửi đồng thời đến tất cả peers.

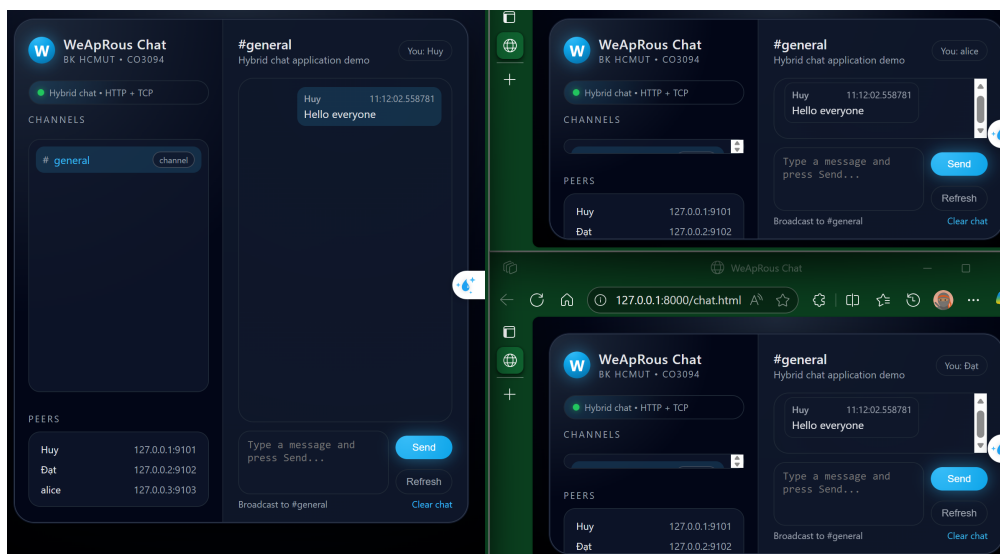


Figure 5: A gửi tin nhắn cho tất cả các peers

6 Kết luận

Bài tập lớn giúp sinh viên nắm rõ cách vận hành hệ thống Web ở mức thấp:

- Hiểu rõ HTTP hoạt động qua TCP
- Tự phân tích request thay vì dùng framework
- Hiểu Cookie và session hoạt động thế nào
- Biết cách xây static file engine
- Triển khai P2P chat qua TCP
- Nắm vững kiến trúc hybrid WebApp